PIC16F72 Microcontroller Simulator: CORE

Nov 4, 2026 · @prakrithi

Overview

Weeks 2-3 deliver a working core for the PIC16F72 simulator: a CPU that runs a fetch, decode, execute, status-update cycle over 8 implemented instructions, plus a text loader that turns assembly-style source into instruction objects.

Week 4 adds `pic_cpu_backend.py`, a Python backend that runs the same CPU as a separate process and grows the instruction set from 8 to 15.

Files

File	Role	Key behavior
`Main.java`	Entry point	Delegates to `SimulatorUI.main`.
`CPU.java`	Processor state and cycle	Holds W register, 128-byte memory, PC, Z and C flags, halted state and a trace log. `step()` runs fetch, decode, execute, `updateStatus()` and returns a formatted trace of W, memory and PC changes. `fetch()` returns the instruction at PC and increments PC; running off the program end halts the CPU.
`Instruction.java`	Abstract base class	Stores `name` and `operand`; subclasses implement `execute(CPU)`. `toDisplayString()` formats the instruction for the UI.
`ProgramLoader.java`	Text-to-program parser	Strips `;` comments and blank lines, uppercases the opcode, parses one integer operand and builds the matching instruction. Unknown opcodes throw `IllegalArgumentException`. Includes a 10-line demo program.
`Movlw.java`	MOVLW k	Loads a literal into W, masked to 8 bits.

File	Role	Key behavior
`Movwf.java`	MOVWF f	Copies W into `memory[f]`.
`Addlw.java`	ADDLW k	W = W + k, masked to 8 bits; C flag set on overflow above 255.
`Sublw.java`	SUBLW k	W = k - W, masked to 8 bits; C flag set when the result is not negative.
`Andlw.java`	ANDLW k	W = W AND k.
`Incf.java`	INCF f	Increments `memory[f]`, wrapping at 255.
`Goto.java`	GOTO addr	Sets PC to the target address.
`Halt.java`	HALT	Sets `halted`, stopping execution.

Design and status

The design uses one class per instruction (polymorphism over `Instruction`), so adding an opcode means one new class plus one `case` in `ProgramLoader`. The CPU advances PC before executing, which lets `Goto` simply overwrite it. Each instruction records `lastResult`, and `updateStatus()` derives the Z flag from it after every step. The demo program exercises all 8 instructions, including a GOTO that skips a `MOVLW 99`.

Known gaps to address next:

- Operands are not range-checked, so an address above 127 throws `ArrayIndexOutOfBoundsException`, and a non-numeric operand throws `NumberFormatException`.
- `Goto` and `Halt` do not update `lastResult`, so Z is recomputed from the previous result.
- Z is updated after every instruction, while on the real PIC16F72 MOVLW and MOVWF leave it unchanged.
- INCF always writes to the file register; the destination bit is not modeled.
- Only 8 instructions exist and no labels are supported; GOTO takes a numeric address.

Week 4: Python backend

`pic_cpu_backend.py` (about 200 lines) moves the CPU into a standalone server that the UI talks to over a socket, and extends the simulator.

Area	What it does
Protocol	TCP server on `127.0.0.1:45772`. Commands: `LOAD` (source lines ending with `END`), `STEP`, `RESET`, `QUIT`. Each reply ends with `---END---`.
Shared memory	Publishes W, PC, Z, C, halted and memory addresses 16-31 as 21 little-endian ints in `pic16f72_sim_shared`, refreshed after every command.
Concurrency	One accept loop, one thread per client, one executor thread fed by a command queue (size 16), and one event thread. A semaphore guards CPU state.
Logging	Events (`COMMAND`, `LOADED`, `EXECUTED`, `HALTED`, `ERROR`) are printed and sent to a logger process on port 45773. The event queue (size 16) drops messages when full.
Parser	Adds labels (with duplicate and invalid-name checks), comma-separated operands, hex or decimal numbers, and a 256-instruction limit.
Validation	Memory addresses must be 0-127 and literals 0-255, so loading bad code returns an error instead of crashing.
New instructions	XORLW, XORWF, RLF, DECFSZ, BTFSC, BTFSS, CLRC. The two skip instructions test only the C flag.

The step trace keeps the Java format and now also reports C flag changes. The week 2-3 gaps on operand range checks and non-numeric operands are fixed in the parser.

Remaining gaps:

- Z is still derived from the last result after every instruction, unlike the real PIC16F72.

- INCF, XORWF and RLF always write back to the file register; the destination bit is not modeled.
- GOTO targets are not range-checked at load time; a bad target halts the CPU on the next step.
- A full command queue blocks the client thread rather than rejecting the command.